

Activity: Git Fundamentals

Objective: To introduce participants to basic Git commands and workflows by creating a simple website and managing its version control.

Part 1: Setting Up Git and Basic Configuration

Check Git Version:

- Instruction: Open your terminal/command prompt and type `git --version`.
- Explanation: This verifies Git is installed and shows its version.
- Participant Action: Run the command and note the output.

Configure User Information:

- Instruction: Run the following commands, replacing the placeholders with your name and email:
 - `git config --global user.name "Your Name"`
 - `git config --global user.email "your.email@example.com"`
- Explanation: This sets your identity for Git commits.
- Participant Action: Run the commands and verify the configuration using `git config --list`.

Initialize a Git Repository:

- Instruction: Create a new folder named "my-website" on your desktop. Navigate into this folder using the terminal/command prompt. Then, run `git init`.
- Explanation: This creates a new Git repository in the current directory.
- Participant Action: Create the folder, navigate to it, run the command. You should see a message indicating the repository was initialized.

Part 2: Creating and Managing Files

Create an HTML File:

- Instruction: Create a file named `index.html` in the "my-website" folder. Add some simple HTML content (e.g., a heading and a paragraph).

Sample Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>My Simple Website</title>
</head>
<body>
  <h1>Welcome to My Website!</h1>
  <p>This is a simple paragraph to demonstrate basic HTML.</p>
</body>
</html>
```

- Participant Action: Create the file and add content.

Check Git Status:

- Instruction: Run `git status`.
- Explanation: This shows the current state of your working directory and staging area.
- Participant Action: Run the command and observe the output. You should see `index.html` as an untracked file.

Add Files to the Staging Area:

- Instruction: Run `git add index.html`.
- Explanation: This stages the `index.html` file for commit.
- Participant Action: Run the command and check `git status` again. You should see `index.html` as a staged file.

Interactive Add (`git add -i`):

- Instruction: Create another file called `style.css` and add some simple CSS content. Run `git add -i`.

Sample Code :

```
body {  
    font-family: sans-serif;  
    background-color: #f4f4f4;  
    margin: 0;  
    padding: 20px;  
}  
  
h1 {  
    color: #333;  
    text-align: center;  
}  
  
p {  
    color: #666;  
    line-height: 1.6;  
}
```

- Explanation: This opens an interactive prompt allowing you to selectively add files.
- Participant Action: Create the file, run the command, and experiment with the interactive options (e.g., status, update, revert). Stage `style.css` using the interactive mode.

Commit Changes:

- Instruction: Run `git add .` to stage all remaining changes. Then, run `git commit -m "Initial commit: website structure"`.

- Explanation: This creates a commit with the staged changes and the provided message.
- Participant Action: Run the commands. You should see a message indicating the commit was successful.

Partial Add (`git add -p`):

- Instruction: Modify `index.html` by adding a new line at beginning and at end.

Sample Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Simple Website</title>
</head>
<body>
  <p>This line was added at the beginning.</p>
  <h1>Welcome to My Website!</h1>
  <p>This is a simple paragraph to demonstrate basic HTML.</p>
  <p>This line was added at the end.</p>
</body>
</html>
```

- Run `git add -p index.html`.
 - Enter “s” to Split the current hunk into smaller hunks
 - When it shows beginning line stage it by entering “y”
 - When it shows end line do not stage it by entering “d”
 - Check status
- Explanation: This allows you to selectively stage parts of a file.
- Participant Action: Modify the file, run the command, and use the interactive prompts to stage only the new line at beginning.

Unstage a File:

- Instruction: modify any line in `style.css`
 - Run `git add style.css` to add it to staging area and check status
 - Run `git restore --staged style.css` to remove it from staging area and check status
- Explanation: This removes `style.css` from the staging area.
- Participant Action: Run the command and check `git status.style.css` should be untracked again.

Part 3: Ignoring Files and Committing Changes

Create a `.gitignore` File:

- Instruction: Create a file named `.gitignore` in the "my-website" folder. Add the line `temp/` to it. Then, create a folder named `temp` and a file named `temp/temp.txt`.
- Explanation: This tells Git to ignore files and folders matching the specified patterns.
- Participant Action: Create the files and folder. Run `git status.temp/` and `temp.txt` should not be listed.
- Run:
`git status -ignored`
- Output:
Ignored files:
(use "`git add -f <file>...`" to include in what will be committed)
`temp/`
- This explicitly shows that Git is ignoring the folder.

Commit Changes (Without `-m`):

- Instruction: Run `git add .` to stage all remaining changes. Then, run `git commit`.
- Explanation: This will open the default text editor configured for Git, allowing you to write the commit message.

- Participant Action: Run the commands. The editor will open. Write a commit message (e.g., "Initial commit: website structure and styling"). Save and close the editor.

Modify the previous commit:

- Instruction: Add a new CSS rule to `style.css`. For example, add `h2 { color: blue; }`.
 - Run `git add style.css`.
 - Run `git commit --amend`.
- Explanation: This will open the editor again, allowing you to modify the previous commit message or add the staged changes to the previous commit.
- Participant Action: The editor will open. You can either:
 - Keep the existing commit message as is (if you only want to add the staged changes).
 - Modify the commit message to reflect the new CSS rule you added. Save and close the editor.

Part 4: Review using git log

Review Commit History (Basic):

- Instruction: Run `git log`.
- Explanation: This shows the full commit history, including commit hashes, authors, dates, and commit messages.
- Participant Action: Run the command and observe the output.

One-Line Commit History:

- Instruction: Run `git log --oneline`.
- Explanation: This displays a condensed version of the commit history, showing only the commit hash and the first line of the commit message.
- Participant Action: Run the command and compare it to the output of `git log`.

Log for a Specific File:

- Instruction: Run `git log style.css`.
- Explanation: This shows the commit history specifically for the `style.css` file.
- Participant Action: Run the command and observe the commits related to `style.css`.

Show Changes (Patch):

- Instruction: Run `git log -p`.
- Explanation: This displays the commit history along with the actual changes made in each commit (the diff).
- Participant Action: Run the command and examine the diff for each commit.

Limit the Number of Commits:

- Instruction: Run `git log -n 2`.
- Explanation: This displays only the last 2 commits.
- Participant Action: Run the command and observe the output. Try changing the number to see different results.

Filter by Date:

- Instruction: Run `git log --since="1 day ago"`.
- Explanation: This shows commits made within the last day. You can adjust the date range (e.g., "1 week ago", "2023-10-26").
- Participant Action: Run the command and experiment with different date ranges.

Modified Part 5: Git Reset and Git Diff

Create a New File:

- Instruction: Create a new file named `script.js` in the "my-website" folder. Add some simple JavaScript code to it (e.g., `console.log("Hello from script.js");`).
- Participant Action: Create the file and add JavaScript content.

Stage and Commit the New File:

- Instruction: Run `git add script.js`. Then, run `git commit -m "Add script.js file"`.
- Participant Action: Run the commands. A new commit will be created.

Soft Reset:

A soft reset in Git moves the HEAD (current branch pointer) to a previous commit, but keeps all your changes staged (in the staging area/index).

- Instruction: Run `git reset --soft HEAD~1`.
- Explanation: This moves the HEAD pointer back by one commit, but keeps the staging area and working directory as they are.
- HEAD~1 refers to One commit before HEAD
- Participant Action: Run the command. Then, run `git status`. Observe that the changes are still staged (the changes that were committed in the previous step).

Note : run `commit -m "JS file added"` to continue the activities.

Mixed Reset:

A mixed reset is the default type of git reset. It moves the HEAD to a previous commit and unstages the changes, but keeps the changes in your working directory.

- Instruction: Run `git reset --mixed HEAD~1`.
- Explanation: This moves the HEAD pointer back by one commit and unstages the changes, but keeps them in the working directory.
- Git performed **three actions**:
- Action 1: Moved HEAD back

From:
Initial commit



Add script.js file ← HEAD

To:

Initial commit ← HEAD

The commit "Add script.js file" no longer exists in the branch history.

Action 2: Removed script.js from Git's index (staging area)
Since `--mixed` resets the index, Git unstaged the file.

Action 3: Left the file on disk
Git did not delete script.js.
It still exists inside your project folder.

- Participant Action: Run the command. Then, run `git status`. Observe that the changes are no longer staged.

Hard Reset:

`git reset --hard` only affects files that Git is tracking. It does not remove untracked files.

- Instruction: Run `git reset --hard HEAD~1`.
- Explanation: This moves the HEAD pointer back by one commit and discards all changes in the staging area and working directory.
- Participant Action: Run the command. Then, run `git status`. Observe that the changes are completely gone.

Modify `index.html` (Unstaged Changes):

- Instruction: Add another new line to `index.html`.
- Participant Action: Modify the file.

View Unstaged Changes:

- Instruction: Run `git diff`.
- Explanation: This shows the changes in the working directory that are not yet staged.
- Participant Action: Run the command and observe the output.

Stage the Changes:

- Instruction: Run `git add index.html`.
- Participant Action: Run the command.

View Staged Changes:

- Instruction: Run `git diff --staged`.
- Explanation: This shows the changes that are staged for commit.
- Participant Action: Run the command and observe the output.

View All Changes (Working Directory vs. HEAD):

- Instruction: Run `git diff HEAD`.
- Explanation: This shows all changes in the working directory compared to the last commit.
- Participant Action: Run the command and observe the output.

View Changes Between Two Commits:

- Instruction: Run `git log --oneline` to get the commit hashes. Use the hash of the commit before the "Add script.js file" commit and the "Add script.js file" commit. Then, run `git diff <commit1> <commit2>`.
- Explanation: This shows the changes between the two specified commits.
- Participant Action: Run the commands, replacing `<commit1>` and `<commit2>` with the actual commit hashes.

View Changes to a Specific File Between Two Commits:

- Instruction: Run `git diff <commit1> <commit2> index.html`.
- Explanation: This shows the changes to `index.html` between the two specified commits.
- Participant Action: Run the command.

View Changes to a Specific File Against a Specific Commit:

- Instruction: Run `git diff HEAD~1 -- index.html`.
- Explanation: This shows the changes to `index.html` in the working directory compared to the version in the previous commit.

- Participant Action: Run the command.

Modify `script.js` (Uncommitted Changes):

- Instruction: Add a new line of JavaScript code to `script.js`.
- Participant Action: Modify the file.

Stash the Changes:

- Instruction: Run `git stash save "Temporary changes to script.js"`.
- Explanation: This saves the uncommitted changes to a new stash.
- Participant Action: Run the command.
-

Check Git Status:

- Instruction: Run `git status`.
- Explanation: This shows the state of the working directory after stash.
- Participant Action: Run the command and observe the output.

Verify Stash List:

- Instruction: Run `git stash list`.
- Explanation: This shows the list of stashed changes.
- Participant Action: Run the command and observe the output.

Modify `index.html` (More Uncommitted Changes):

- Instruction: Add another new line to `index.html`.
- Participant Action: Modify the file.

Apply the Stash:

- Instruction: Run `git stash apply`.
- Explanation: This applies the most recent stash to the working directory.
- Participant Action: Run the command.

Check Git Status:

- Instruction: Run `git status`.
- Explanation: This shows the state of the working directory after applying the stash.
- Participant Action: Run the command and observe the output.